

Guia Completo: Câmera 3D e Movimentação

Explicação didática da matemática, variáveis do coelho, visão panorâmica orbital e visão em primeira pessoa no OpenGL.

1. A Grande Analogia: Como o OpenGL Enxerga a Câmera?

No pipeline clássico do OpenGL, **não existe um objeto físico palpável chamado "câmera"** que você coloca dentro do cenário. O mundo 3D é apenas um aglomerado de pontos matemáticos (vértices).

Para enxergar esse mundo, pense que o OpenGL é um **cinegrafista segurando uma filmadora**. A cada quadro desenhado na tela (cerca de 40 a 60 vezes por segundo), você precisa dar exatamente **3 ordens** para esse cinegrafista através da função `gluLookAt`:

As 3 Perguntas do Cinegrafista:

1. **Onde você quer que eu fique em pé?** (Posição dos Olhos / *Eye*: `eyeX, eyeY, eyeZ`)
2. **Para onde você quer que a lente aponte?** (Ponto de Foco / *Target*: `targetX, targetY, targetZ`)
3. **A câmera está reta ou de ponta-cabeça?** (Vetor Vertical / *Up Vector*: `upX, upY, upZ` — geralmente `0, 1, 0` , dizendo que o eixo Y aponta para cima).

```
gluLookAt(  
    eyeX, eyeY, eyeZ,          // 1. Onde a lente está posicionada no espaço  
    targetX, targetY, targetZ, // 2. Qual ponto do mundo a lente está mirando  
    upX, upY, upZ             // 3. Vetor que aponta "para o teto" (0.0, 1.0, 0.0)  
);
```

2. Dicionário Completo de Variáveis

Abaixo estão todas as variáveis que controlam o coelho e as câmeras no projeto:

Variável	Tipo	Arquivo	Papel no Jogo
<code>coelhoX, coelhoY, coelhoZ</code>	<code>float</code>	<code>Coelho3D.hpp</code>	Posição tridimensional do centro do coelho no mundo. <code>Y</code> é a altura (varia com o pulo).
<code>anguloCoelho</code>	<code>float</code>	<code>Coelho3D.hpp</code>	Direção horizontal em que o coelho está virado (em graus). <code>0°</code> = olhando para o eixo <code>+X</code> .
<code>characterSpeed</code>	<code>float</code>	<code>Coelho3D.hpp</code>	Velocidade linear de corrida do coelho pelo gramado.
<code>cameraPrimeiraPessoa</code>	<code>bool</code>	<code>main.cpp</code>	Alternador de modo: <code>false</code> = Visão Geral Panorâmica; <code>true</code> = Visão em 1ª Pessoa (tecla C).
<code>anguloCameraGeral</code>	<code>float</code>	<code>main.cpp</code>	Ângulo orbital da câmera geral ao redor do eixo Y do campo (tecla R).
<code>RAIO_CAMERA_GERAL</code>	<code>const float</code>	<code>main.cpp</code>	Distância horizontal da câmera geral em relação ao centro (fixado em <code>10.0f</code>).
<code>ALTURA_CAMERA_GERAL</code>	<code>const float</code>	<code>main.cpp</code>	Altura fixa da câmera geral acima do chão (fixado em <code>10.0f</code>).
<code>pitchOlharPrimeiraPessoa</code>	<code>float</code>	<code>main.cpp</code>	Inclinação vertical da visão em 1ª pessoa (olhar para o céu ou para o chão, teclas W/S).
<code>CAMERA_FOV</code>	<code>const double</code>	<code>main.cpp</code>	Abertura angular da lente (Field of View = <code>55.0°</code>), configurada no <code>gluPerspective</code> .

3. Como o Coelho se Movimenta no Chão?

No `Coelho3D.cpp`, a função `moverCoelho()` roda a cada frame e traduz as teclas de seta no comportamento físico do personagem:

A. Girando no Próprio Eixo (Setas Esquerda e Direita)

As setas para os lados **não** deslocam o coelho de posição. Elas apenas giram a orientação do corpo:

- **Seta Esquerda:** aumenta `anguloCoelho` (gira no sentido anti-horário visto de cima).
- **Seta Direita:** diminui `anguloCoelho` (gira no sentido horário).

B. Calculando a Direção para Frente (Vetor Frontal)

Para andar para frente ou para trás, o coelho precisa saber para onde o nariz dele está apontando. Usamos trigonometria básica:

```
float rad = anguloCoelho * PI_F / 180.0f;
float fx = std::cos(rad); // Projecção no eixo X
float fz = -std::sin(rad); // Projecção no eixo Z (-Z vai para o fundo)
```

C. Deslocamento Linear (Setas Cima e Baixo)

Quando a seta para cima é pressionada, o coelho soma esse vetor frontal na sua posição atual:

```
coelhoX += fx * (deslocamentoLinear * characterSpeed);
coelhoZ += fz * (deslocamentoLinear * characterSpeed);
```

D. O Pulo (Barra de Espaço)

Ao apertar espaço, a função `atualizarPulo()` faz o valor de `coelhoY` crescer suavemente até a altura máxima e depois cair até tocar a grama (`coelhoY = 0.5f`).

4. Modo 1: Câmera em Visão Geral (Modo Drone / Terceira Pessoa)

Quando `cameraPrimeiraPessoa == false`, o jogo utiliza uma câmera elevada e panorâmica para permitir que o jogador veja todo o campo tático.

Posição Orbital Dinâmica (Tecla R)

O enunciado exige que a posição inicial seja `(0, 10, 10)` mirando em `(0, 0, 0)`. Ao apertar a tecla **R**, a câmera descreve um círculo perfeito de raio 10 ao redor do eixo Y:

- `camGeralX = 10.0f * sin(anguloCameraGeral)`
- `camGeralY = 10.0f` (altura constante)
- `camGeralZ = 10.0f * cos(anguloCameraGeral)`

Se `anguloCameraGeral == 0°`: $\sin(0) = 0$ e $\cos(0) = 1$, resultando exatamente em **(0, 10, 10)**!

```
gluLookAt(  
    camGera1X, camGera1Y, camGera1Z, // Cinegrafista voando em órbita circular  
    0.0, 0.0, 0.0, // Mirando fixo no centro do campo  
    0.0, 1.0, 0.0 // Vetor vertical (eixo Y para cima)  
);
```

5. Modo 2: Câmera em Primeira Pessoa (Pelos Olhos do Coelho)

Ao pressionar a tecla **C**, você assume a visão diretamente dos olhos do coelho. Essa é a parte mais fascinante da implementação:

Passo 1: Onde Ficam os Olhos? (Eye Position)

O ponto `(coelhoX, coelhoY, coelhoZ)` é o centro geométrico da barriga do coelho. Se colocássemos a câmera ali, ela ficaria no meio do tronco. Anatomicamente:

- **Altura dos olhos:** A cabeça fica `+0.26` acima do centro do corpo:

```
camOlhoY = coelhoY + 0.26f;
```

Nota crucial: quando o coelho pula (`coelhoY` sobe), os olhos sobem juntos instantaneamente!

- **Avanço para a frente da cabeça:** Os olhos ficam `+0.35` projetados para frente na direção em que o coelho está olhando:

```
camOlhoX = coelhoX + 0.35f * cos(yawRad);
```

```
camOlhoZ = coelhoZ + 0.35f * (-sin(yawRad));
```

Passo 2: Para Onde a Câmera Mira? (Target Position)

A câmera precisa olhar exatamente na mesma direção para onde o coelho está virado (ângulo de yaw), mas com suporte a olhar para cima e para baixo (ângulo de **pitch** pelas teclas **W/S** ou **PgUp/PgDn**):

```
float cosPitch = std::cos(pitchRad);
float dirOlharX = cosPitch * std::cos(yawRad);
float dirOlharY = std::sin(pitchRad); // Positivo = olha pro céu; Negativo = olha pro chão
float dirOlharZ = cosPitch * (-std::sin(yawRad));

// Ponto alvo colocado 5 unidades à frente na direção do vetor:
float alvoOlharX = camOlhoX + dirOlharX * 5.0f;
float alvoOlharY = camOlhoY + dirOlharY * 5.0f;
float alvoOlharZ = camOlhoZ + dirOlharZ * 5.0f;
```

Chamada final em primeira pessoa:

```
gluLookAt(
    camOlhoX, camOlhoY, camOlhoZ, // Olhos do coelho
    alvoOlharX, alvoOlharY, alvoOlharZ, // Ponto focal à frente
    0.0, 1.0, 0.0 // Cabeça em pé
);
```

⚠ O Detalhe Crucial: Ocultar o Próprio Coelho

Se o jogo continuasse desenhando o modelo 3D do coelho em primeira pessoa, a lente da câmera estaria **dentro do crânio dele**. O jogador só veria polígonos colados na tela (interior dos dentes e bochechas).

Por isso, no `display()` existe a condição:

```
if (!cameraPrimeiraPessoa) { drawRabbit(); }
```

Isso garante que o modelo só seja renderizado quando visto de fora!

6. Comparativo Rápido Entre os Dois Modos

Aspecto	Visão Geral (Panorâmica)	Visão em 1ª Pessoa
Posição (Eye)	Órbita circular <code>(R*sin θ, 10, R*cos θ)</code>	Nos olhos do coelho <code>(coelhoX+dx, coelhoY+0.26, coelhoZ+dz)</code>
Alvo (Target)	Centro da arena <code>(0, 0, 0)</code>	5 metros à frente na linha de visão do coelho
Teclas de Controle	R (girar câmera ao redor da arena)	Setas (andar/virar) e W/S (olhar cima/baixo)
Modelo do Coelho	Visível (desenhado com <code>drawRabbit()</code>)	Oculto (para não tampar a visão do jogador)
Objetivo de Gameplay	Visão tática ampla de raposas e vegetais	Imersão total no nível do chão e caça aérea da ave